# Adaptive Drone Swarm Behavior Simulation

Project Report


**Prepared By:**

Anup Baigain

Sameer Baskota

Madhusudan Pun

Toran Bahadur Niroula


**Submitted To:**

Samsung Innovation Campus

Pulchowk Campus

**Date:**

May 8, 2025

## Abstract

This report covers the design and implementation of a 3D simulation environment for analyzing and visualizing adaptive drone swarm behaviors. The simulation, developed using Python with tkinter for the GUI and matplotlib for 3D visualization, incorporates bio-inspired algorithms. These include Boids behaviors (cohesion, alignment, separation) for flocking, and a Hybrid Control System using PD control for pattern formation and PID control for obstacle avoidance and goal seeking. The platform simulates scenarios with static cylindrical obstacles. A modular and extensible architecture was created for future research, such as integrating reinforcement learning or role-based task allocation. This work applies Python skills from the Samsung Innovation Campus and provides a foundation for understanding multi-agent systems in autonomous aerial robotics.
**keywords:** drones, swarm, simulation, python programming

## Abstract

This report covers the design and implementation of a 3D simulation environment for analyzing and visualizing adaptive drone swarm behaviors. The simulation, developed using Python with `tkinter` for the GUI and `matplotlib` for 3D visualization, incorporates bio-inspired algorithms. These include Boids behaviors (cohesion, alignment, separation) for flocking, and a Hybrid Control System using PD control for pattern formation and PID control for obstacle avoidance and goal seeking. The platform simulates scenarios with static cylindrical obstacles. A modular and extensible architecture was created for future research, such as integrating reinforcement learning or role-based task allocation. This work applies Python skills from the Samsung Innovation Campus and provides a foundation for understanding multi-agent systems in autonomous aerial robotics.
**keywords:** drones, swarm, simulation, python programming

# Contents

**Abstract** **1**

**1 Introduction** **3**

**2 Project Objectives and Scope** **3**
2.1 Core Objectives . . . . . 3
2.2 Scope and Deliverables . . . . . 3

**3 Software Architecture Overview** **4**

**4 Implementation Details** **4**
4.1 Core Classes . . . . . 4
4.2 Swarm Behaviors . . . . . 5
4.3 Advanced Features . . . . . 7
4.4 User Interaction . . . . . 9

**5 Results and Discussion** **9**
5.1 Demonstrated Behaviors . . . . . 9
5.2 Visualization and Analysis . . . . . 10
5.3 Extensibility . . . . . 10

**6 Conclusion** **10**

**A Source Code: drones.py** **12**

# 1 Introduction

Drone swarm technology has potential in domains like environmental monitoring, search and rescue, logistics, and infrastructure inspection. Multiple autonomous vehicles coordinating as one entity offer advantages like improved efficiency, robustness, and scalability over single-agent systems. This project explores collective drone behaviors by developing a 3D simulation. Inspired by bird flocking and fish schooling, the simulation models emergent behaviors using distributed control algorithms. The main objective is to create a Python platform to demonstrate swarm intelligence concepts and support testing of advanced coordination strategies for adaptive drone swarms. This report outlines the design, implementation, and evaluation of this platform.

# 2 Project Objectives and Scope

Core objectives for this project included building a functional and extensible drone swarm simulation platform. The project scope ensured achievable deliverables.

## 2.1 Core Objectives

- **Develop a Modular Simulation Platform:** Create a flexible Python-based environment for diverse swarm behaviors and scenarios.
- **Implement Swarm Behaviors:** Include Boids-inspired decentralized control (cohesion, alignment, separation).
- **Design Collision Avoidance:** Enable drones to detect and avoid static and dynamic obstacles in 3D.
- **Enable Dynamic Pattern Formation:** Allow the swarm to form predefined geometric patterns (e.g., circle, square, line, star, triangle, '1', 'S') and manage transitions.
- **3D Simulation and Advanced Concepts:** Implement in 3D and prepare for future integration of features like reinforcement learning.

## 2.2 Scope and Deliverables

Project scope and deliverables:

- **Drone Movement and Collision Detection:** Implement drone kinematics and collision detection.
- **Swarm Intelligence Algorithms:** Implement and demonstrate cohesion, alignment, and separation rules.
- **Obstacle Avoidance:** Develop algorithms for navigating around static obstacles and avoiding drone collisions.
- **Pattern Formations:** Implement functions for generating pattern targets and drone transitions.
- **Visualization and GUI:** Create a `tkinter` GUI to control parameters and visualize the swarm in 3D with `matplotlib`.
- **Documentation and Analysis:** Provide code documentation and analyze simulation results (e.g., success rates, completion times).

# 3 Software Architecture Overview

The platform uses a modular architecture for flexibility and maintainability. Key modules include:

- **Physics Engine:** Simulates drone movement, calculates position changes based on velocity and forces, and detects collisions.
- **Swarm Behavior Module:** Implements decentralized control (Boids-inspired rules for cohesion, alignment, separation) and logic for obstacle avoidance and goal-seeking, often using PID control.
- **Visualization System:** Manages graphical representation using `tkinter` for the UI and `matplotlib` for 3D real-time visualization of drones, obstacles, and boundaries.
- **Scenario Manager:** Sets up the simulation environment, including boundaries, initial drone positions, obstacle locations and dimensions, and target positions for pattern formations.

# 4 Implementation Details

This section details the Python implementation of core components and behaviors.

## 4.1 Core Classes

Key Python classes define simulation elements:

- **Pillar:** Represents a static cylindrical obstacle. Stores base position, height, and radius. Its `is_inside` method checks if a point is within the pillar's boundaries for collision detection.
- **Drone:** Represents an autonomous agent. Maintains ID, 3D position, velocity, target position, maximum speed, and control parameters (PID gains, Boids strengths). Methods calculate forces acting on the drone. The `update` method integrates these forces to update position and velocity.
- **Swarm:** Orchestrates the simulation. Manages drone objects, pillar obstacles, and the simulation state (mode, time). Initializes the swarm and obstacles, sets pattern targets, updates drones, calculates results in obstacle avoidance mode, and handles mode transitions.
- **SimulatorGUI:** Provides the `tkinter`-based user interface. Allows user control over simulation mode, drone count, pattern selection, PID gains, simulation speed, and simulation lifecycle (start, stop, reset). Integrates the `matplotlib` plot for real-time 3D visualization.

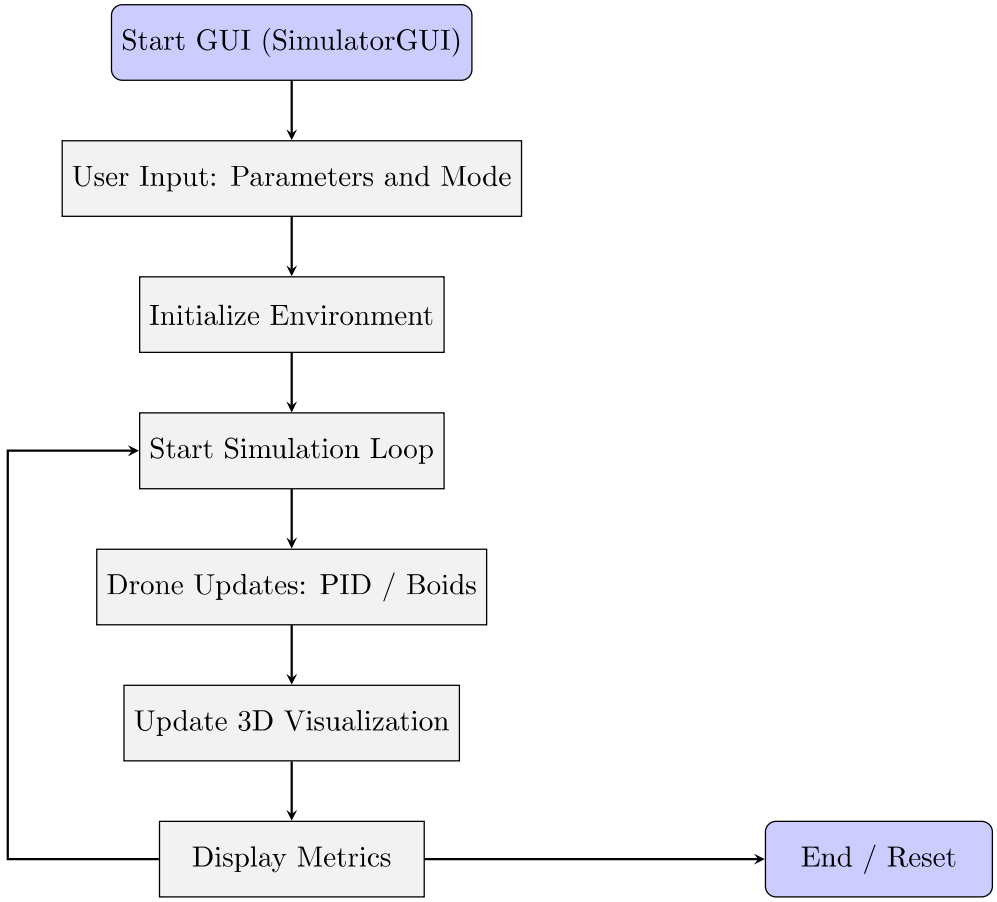



Figure 1: Flowchart of the drone swarm simulation architecture showing GUI interaction, control loop, drone updates, visualization, and feedback.

### 4.2 Swarm Behaviors

Swarm behavior results from algorithms in the `Drone` and `Swarm` classes:

- **Cohesion, Alignment, and Separation (Boids):** Implemented in the Drone class.

```
FUNCTION calculate_boids_forces(current_drone, all_drones_in_swarm)
  INITIALIZE separation_force = [0,0,0]
  INITIALIZE neighbor_positions_list, neighbor_velocities_list
  INITIALIZE swarm_center_accumulator = [0,0,0], neighbor_count = 0

  FOR each other_drone in all_drones_in_swarm DO
    IF other_drone is not current_drone AND other_drone is active THEN
      CALCULATE distance_to_other_drone
      IF distance_to_other_drone < current_drone.perception_radius THEN
        ADD other_drone.position to neighbor_positions_list
        ADD other_drone.velocity to neighbor_velocities_list
        ADD other_drone.position to swarm_center_accumulator
        INCREMENT neighbor_count
        IF distance_to_other_drone > very_small_threshold THEN
          CALCULATE repel_direction from other_drone
          ADD (repel_direction * strength / distance_to_other_drone^2)
            to separation_force
        END IF
      END IF
    END IF
  END FOR
```

```
  INITIALIZE cohesion_force = [0,0,0], alignment_force = [0,0,0]
  IF neighbor_count > 0 THEN
    average_position = swarm_center_accumulator / neighbor_count
    cohesion_force = (average_position - current_drone.position)
                   * cohesion_strength

    average_velocity = MEAN(neighbor_velocities_list)
    IF magnitude of average_velocity > very_small_threshold THEN
      alignment_force = (normalize(average_velocity) -
                         normalize(current_drone.velocity))
                        * alignment_strength
    END IF
  END IF
  RETURN separation_force, cohesion_force, alignment_force
END FUNCTION
```

Listing 1: Boids Calculation Simplified

- **Pattern Formation:** Drones form predefined 3D patterns.

```
FUNCTION generate_pattern_targets(pattern_name, pattern_size, all_drones,
    area_dims)
  INITIALIZE target_points_list
  active_drones = FILTER active drones from all_drones
  num_active = COUNT active_drones

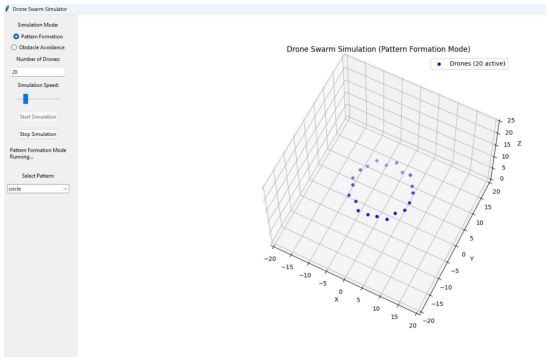
  IF num_active IS 0 THEN RETURN empty_list

  center_point = CALCULATE center of pattern area (e.g., [0,0, Z_offset])

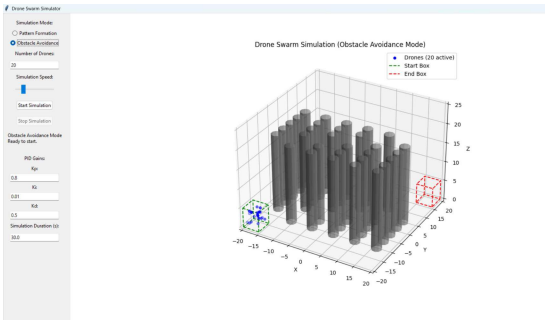
  SWITCH pattern_name:
    CASE "circle":
      FOR i from 0 to num_active-1:
        CALCULATE x,y,z for point i on circle perimeter
        ADD [x,y,z] + center_point to target_points_list
      END FOR
    CASE "square", "line", "triangle", etc.:
      // ... Logic to calculate points for the specific pattern ...
      // Add points + center_point to target_points_list
  END SWITCH

  final_drone_targets = INITIALIZE empty list
  target_idx = 0
  FOR each drone in all_drones (maintaining order for assignment):
    IF drone is active:
      IF target_idx < COUNT(target_points_list):
        ADD target_points_list[target_idx] to final_drone_targets
        INCREMENT target_idx
      ELSE: // Not enough calculated points, drone keeps position
        ADD drone.current_position to final_drone_targets
      END IF
    END IF
  END FOR
  RETURN final_drone_targets
END FUNCTION
```

Listing 2: Generating Pattern Targets conceptual




(a) Example of a circular pattern formation




(b) Example of pillar obstacles in the simulation

Figure 2: Visualization of key simulation elements

- **Obstacle Avoidance and Goal Seeking:** Drones navigate obstacles to a goal.

```
// For Pillar Class
FUNCTION is_point_inside_pillar(pillar_props, point_pos)
  dx = point_pos.x - pillar_props.base_x
  dy = point_pos.y - pillar_props.base_y
  dist_sq_to_axis = dx*dx + dy*dy

  within_radius = dist_sq_to_axis <= pillar_props.radius^2
  within_height = (point_pos.z >= pillar_props.base_z AND
                   point_pos.z <= pillar_props.base_z +
                       pillar_props.height)
  RETURN within_radius AND within_height
END FUNCTION

// For Drone update logic (obstacle avoidance mode)
PROCEDURE check_drone_obstacle_collision(drone, all_obstacles)
  FOR each obstacle in all_obstacles DO
    IF is_point_inside_pillar(obstacle.properties, drone.position) THEN
      SET drone.is_eliminated = TRUE
      BREAK // Collision detected
    END IF
  END FOR
END PROCEDURE
```

Listing 3: Obstacle Collision Check

## 4.3 Advanced Features

Advanced features enhance realism and allow for future expansion:

- **Hybrid Control System:** The simulation switches control strategies by mode.

```
PROCEDURE calculate_total_force_on_drone(drone, swarm_ctx, time_delta)
  INITIALIZE total_force_vector = [0,0,0]

  IF swarm_ctx.mode IS 'pattern_formation' THEN
    pattern_force = drone.calc_pattern_following_force()
    ADD pattern_force to total_force_vector

    inter_drone_avoid =
        drone.calc_separation_from_neighbors(swarm_ctx.drones)
    ADD inter_drone_avoid to total_force_vector
```

```
  ELSE IF swarm_ctx.mode IS 'obstacle_avoidance' THEN
    goal_seek_force = drone.calc_pid_to_goal(swarm_ctx.goal, time_delta)
    ADD goal_seek_force to total_force_vector

    obstacle_avoid = drone.calc_obstacle_repulsion(swarm_ctx.obstacles)
    ADD obstacle_avoid to total_force_vector

    boids_forces (sep,coh,align) =
        drone.calc_boids_with_neighbors(swarm_ctx.drones)
    ADD boids_forces to total_force_vector
  END IF
  RETURN total_force_vector
END PROCEDURE
```

Listing 4: Hybrid Control Logic (Conceptual in Drone Update)

- **Adaptive PID Controller:** PID gains (Kp, Ki, Kd) are tunable via GUI.

```
FUNCTION calculate_pid_force(drone, goal_pos, time_delta)
  error = goal_pos - drone.current_position

  P_term = drone.Kp_gain * error

  drone.integral_accumulator = drone.integral_accumulator +
                               (error * time_delta)
  I_term = drone.Ki_gain * drone.integral_accumulator

  IF time_delta > 0 THEN
    derivative_error = (error - drone.previous_pid_error) / time_delta
  ELSE
    derivative_error = [0,0,0]
  END IF
  D_term = drone.Kd_gain * derivative_error
  drone.previous_pid_error = error

  pid_combined_force = P_term + I_term + D_term

  dist_to_goal = MAGNITUDE(error)
  IF dist_to_goal < drone.arrival_radius AND
     MAGNITUDE(drone.velocity) < 0.1 THEN
    RETURN [0,0,0]
  END IF

  IF MAGNITUDE(pid_combined_force) > drone.max_allowable_force THEN
    pid_combined_force = NORMALIZE(pid_combined_force) *
                         drone.max_allowable_force
  END IF
  RETURN pid_combined_force
END FUNCTION
```

Listing 5: PID Force Calculation

- **Real-time 3D Visualization:** `matplotlib` provides 3D visualization.

```
PROCEDURE plot_swarm_state(gui, swarm_obj)
  CLEAR gui.plot_axis

  IF gui.plot_axis is not 3D_enabled THEN
    INITIALIZE gui.plot_axis as 3D_subplot
  END IF
```

```
  active_list = GET_ACTIVE_DRONES(swarm_obj.drones)
  IF active_list is not empty:
    positions = EXTRACT_POSITIONS(active_list)
    PLOT_SCATTER(gui.plot_axis, positions, color='blue', label='Active')
  END IF

  eliminated_list = GET_ELIMINATED_DRONES(swarm_obj.drones)
  IF eliminated_list is not empty:
    positions = EXTRACT_POSITIONS(eliminated_list)
    PLOT_SCATTER(gui.plot_axis, positions, color='red', label='Eliminated')
  END IF

  reached_list = GET_REACHED_DRONES(swarm_obj.drones)
  IF reached_list is not empty:
    positions = EXTRACT_POSITIONS(reached_list)
    PLOT_SCATTER(gui.plot_axis, positions, color='green', label='Reached')
  END IF

  IF swarm_obj.mode IS 'obstacle_avoidance':
    FOR each obstacle in swarm_obj.obstacles:
      mesh_data = CALCULATE_CYLINDER_MESH(obstacle.properties)
      PLOT_SURFACE(gui.plot_axis, mesh_data, color='gray', alpha=0.6)
    END FOR
  END IF

  SET_PLOT_LABELS(gui.plot_axis, "X", "Y", "Z")
  SET_PLOT_TITLE(gui.plot_axis, "Drone Swarm - Mode: " + swarm_obj.mode)
  SHOW_LEGEND(gui.plot_axis)
  REDRAW_CANVAS(gui.plot_canvas)
END PROCEDURE
```

Listing 6: 3D Plotting Logic (Simplified)

### 4.4 User Interaction

The GUI gives users control over the simulation. Key functionalities include:

- **Mode Switching:** Switch between "Pattern Formation" and "Obstacle Avoidance" modes using radio buttons.
- **Swarm Configuration:** Adjust the number of drones. In pattern formation, select the pattern from a dropdown menu.
- **Parameter Tuning:** In obstacle avoidance, input and update Kp, Ki, Kd gains for the PID controller. Set simulation duration.
- **Simulation Control:** Buttons to start, stop, and reset the simulation.
- **Real-time Feedback:** An information label displays current mode and, in obstacle avoidance, results of the last run (successful/eliminated drones, average goal time).

## 5 Results and Discussion

The simulation demonstrates emergent swarm behaviors and allows analysis of their effectiveness.

### 5.1 Demonstrated Behaviors

- **Emergent Swarm Behavior:** The simulation shows flocking due to cohesion, alignment, and separation rules. Drones maintain group integrity and avoid collisions.

- **Dynamic Pattern Formation:** The mode shows the swarm forming geometric shapes. Smooth transitions demonstrate dynamic capabilities.
- **Adaptive Navigation and Obstacle Avoidance:** The swarm navigates static pillars. PID goal seeking and repulsion forces enable pathfinding.

### 5.2 Visualization and Analysis

- **Informative 3D Visualization:** The real-time 3D plot offers insights into swarm spatial behavior. Visualization aids in understanding algorithm effectiveness. The visualization uses standard colors for different drone states (active, eliminated, reached goal) and obstacles for clarity.
- **Performance Metrics:** Metrics like success rate and average goal time offer quantitative data for evaluating swarm efficiency. This data allows comparison of different control parameters or algorithms.

### 5.3 Extensibility

The modular codebase design allows for future extensions. Potential future enhancements include:

- **Reinforcement Learning Integration:** The framework can be used to explore reinforcement learning for navigation or pattern switching.
- **Adaptive Formation Switching:** Implement logic for autonomous pattern selection based on cues or requirements.
- **Role-Based Tasking:** Assign roles for complex coordinated tasks.
- **More Complex Environments:** Add dynamic obstacles or environmental factors (e.g., wind).

## 6 Conclusion

This project developed a functional 3D simulation environment for adaptive drone swarm behaviors, meeting its objectives. The implementation of Boids-inspired rules, a hybrid control system, and real-time 3D visualization serves as a tool for demonstrating and analyzing swarm intelligence. The project applied Python programming skills from the Samsung Innovation Campus and built a foundation for future research in multi-agent systems and autonomous aerial robotics. The modular architecture allows the simulation to be extended for more advanced algorithms and complex scenarios. Developing this simulation was a learning experience beyond algorithm implementation. It helped in understanding challenges in designing and simulating multi-agent systems.

- **Understanding Emergence:** Working with the Boids model and decentralized control showed how complex behavior can emerge from simple local rules and agent interactions.
- **Problem Solving and Iteration:** The process involved designing class structures, implementing physics, tuning control parameters, and debugging the visualization.
- **Team Collaboration:** Working on different simulation modules fostered teamwork and communication.
- **Connecting Theory to Practice:** The project linked theoretical concepts from the Samsung Innovation Campus program to practical implementation in object-oriented programming, control systems, and simulation techniques.

## References

[1] SwarmLab: A MATLAB Drone Swarm Simulator. https://arxiv.org/pdf/2005.02769

[2] Tkinter Documentation. https://docs.python.org/3/library/tkinter.html

[3] Matplotlib Documentation. https://matplotlib.org/stable/index.html

[4] LIS EPFL Swarm Lab. https://github.com/lis-epfl/swarmlab

# A Source Code: drones.py

```
import tkinter as tk
from tkinter import ttk
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import time
import threading
import math
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg


class Pillar:
    """Represents a cylindrical pillar obstacle in the simulation."""
    def __init__(self, base_position, height, radius):
        self.base_position = np.array(base_position, dtype=float)
        self.height = height
        self.radius = radius

    def is_inside(self, position):
        """Checks if a given 3D position is inside the pillar."""
        dx = position[0] - self.base_position[0]
        dy = position[1] - self.base_position[1]
        dist_sq = dx**2 + dy**2
        is_within_radius = dist_sq <= self.radius**2
        is_within_height = (position[2] >= self.base_position[2] and
                            position[2] <= self.base_position[2] + self.height)
        return is_within_radius and is_within_height


class Drone:
    """Represents an individual drone in the swarm."""
    def __init__(self, drone_id, position, velocity=None):
        self.id = drone_id
        self.position = np.array(position, dtype=float)
        self.velocity = np.array(velocity if velocity is not None else [0.0,
    0.0, 0.0], dtype=float)
        self.target_position = np.array(position, dtype=float)
        self.max_speed = 4.0
        self.arrival_threshold_pattern = 0.2

        self.Kp_pattern = 0.5
        self.Kd_pattern = 0.5

        self.goal_seeking_strength_boids = 0.4
        self.cohesion_strength = 0.1
        self.alignment_strength = 0.1
        self.avoidance_radius = 3.0
        self.repulsion_strength = 2.5

        self.Kp_pid = 0.8
        self.Ki_pid = 0.01
        self.Kd_pid = 0.5
        self.integral_error = np.array([0.0, 0.0, 0.0])

        self.arrival_threshold_goal = 2.0

        self.eliminated = False
        self.reached_goal = False
        self.time_to_goal = -1.0

        self.path_taken = []
```

```
        self.broadcast_info = {}

        self.previous_error = np.array([0.0, 0.0, 0.0])

    def calculate_pattern_force(self):
        """Calculates the force to move the drone towards its target_position in pattern formation mode."""
        position_error = self.target_position - self.position
        velocity_error = np.array([0.0, 0.0, 0.0]) - self.velocity

        pattern_force = self.Kp_pattern * position_error + self.Kd_pattern * velocity_error

        distance_to_target = np.linalg.norm(position_error)
        if distance_to_target < self.arrival_threshold_pattern and np.linalg.norm(self.velocity) < 0.1:
            return np.array([0.0, 0.0, 0.0])

        force_magnitude = np.linalg.norm(pattern_force)
        if force_magnitude > self.max_speed:
            pattern_force = pattern_force / force_magnitude * self.max_speed
        return pattern_force

    def calculate_pid_force(self, goal_position, dt):
        """Calculates force using a PID controller towards the goal_position."""
        error = goal_position - self.position

        P_term = self.Kp_pid * error

        self.integral_error += error * dt
        I_term = self.Ki_pid * self.integral_error

        derivative_error = (error - self.previous_error) / dt if dt > 0 else np.array([0.0, 0.0, 0.0])
        D_term = self.Kd_pid * derivative_error

        self.previous_error = error.copy()

        pid_force = P_term + I_term + D_term

        distance_to_goal = np.linalg.norm(error)
        if distance_to_goal < self.arrival_threshold_goal and np.linalg.norm(self.velocity) < 0.1:
            return np.array([0.0, 0.0, 0.0])

        force_magnitude = np.linalg.norm(pid_force)
        if force_magnitude > self.max_speed:
            pid_force = pid_force / force_magnitude * self.max_speed
        return pid_force

    def calculate_goal_force_boids(self, goal_position):
        """Calculates the force towards a fixed goal_position for Boids-like behavior."""
        direction_to_goal = goal_position - self.position
        distance_to_goal = np.linalg.norm(direction_to_goal)

        if distance_to_goal > self.arrival_threshold_goal:
            goal_force = direction_to_goal / distance_to_goal * self.goal_seeking_strength_boids
            return goal_force
        else:
            return np.array([0.0, 0.0, 0.0])
```